Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 1: Tema - 1. FUNDAMENTOS DE SOFTWARE E GERENCIAMENTO DE PROJETOS

3 Objetivos

Reconhecer a necessidade do desenvolvimento de software ser com base em princípios da engenharia de software.

4 Tópicos

1.1 PROCESSO DE FABRICAÇÃO DO HARDWARE X PROCESSO DE DESENVOLVIMENTO DO SOFTWARE
1.2 ENGENHARIA DE SOFTWARE

5 Procedimentos de ensino-aprendizagem

Nesta aula, após as apresentações iniciais, serão debatidos conceitos introdutórios da disciplina, utilizando-se analogias e exemplos do mundo real.

- Situação-problema:
Imagine que você tem um terreno e deseja construir uma casa. Qual o primeiro passo? Você iniciará pelas paredes, pela fundação ou pelo projeto? E quando você tem um problema e deseja construir um software para solucioná-lo, por onde você deve começar?

- Metodologia:
O professor deve iniciar a aula apresentando as definições de software e sistema juntamente com suas classificações e contextualizar historicamente como era o desenvolvimento dos primeiros programas, e as dificuldades enfrentadas por falta de planejamento. Dificilmente os prazos eram respeitados e quando o software ficava pronto, muitas vezes, ele não era mais necessário. Seguindo neste tema, o professor deve explicar o que foi a Crise do Software e como surgiu a Engenharia de Software. Na sequência pode apresentar o vídeo "O Que faz um Engenheiro De Software" [1], e contextualizar o papel deste profissional no mercado de trabalho. Para complementar, o professor deve realizar uma analogia da fabricação de hardware com o desenvolvimento de software. Pode-se comparar também os seus ciclos de vida. Enquanto o hardware geralmente apresenta mais problemas no final do seu ciclo de vida, após o desgaste dos componentes, o software apresenta problemas no início do seu uso e depois de algumas manutenções alcança a estabilidade. Para finalizar, o professor pode debater com a turma os mitos e verdades sobre desenvolvimento de software, apresentados pelo Pressman [2] no capítulo 1.

- Atividade verificadora de aprendizagem:
Em grupos, pesquisem na internet um caso de um software que tenha fracassado. Identifiquem qual foi o seu principal problema e indiquem como a engenharia de software poderia ter evitado o insucesso. Após a pesquisa os grupos deverão apresentar e debater com toda a turma as suas conclusões.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "O Que faz um Engenheiro De Software" disponível em: https://www.youtube.com/watch?v=ykFwpzab8rM

[2] PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Porto Alegre: AGMH, 2016. Páginas 14 até 27. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788580555349/

8 Aprenda +

Vídeo "Software Engineering: Crash Course Computer Science #16" disponível em: https://www.youtube.com/watch?v=O753uuutqH8 (Acesse usando chrome a ative a legenda, e a tradução automática para português)

Atividade Autônoma Aura:

1) Sobre os mitos e verdades da Engenharia de Software, analise as opções abaixo e marque a alternativa que apresenta uma verdade.
a) Se um projeto estiver atrasado, deve-se solucionar o problema adicionando mais pessoas ao time.
b) A documentação é parte importantíssima do projeto e deve ser elaborada ao fim do desenvolvimento.
c) Uma definição geral dos objetivos é suficiente para iniciar a escrita dos programas.

d) A avaliação da qualidade e dos objetivos deve ocorrer durante todo o projeto.

e) A engenharia de software produz documentação volumosa e desnecessária, além retardar o desenvolvimento

2) Dada a Engenharia de Software, identifique a principal motivação/indicação para seu surgimento:
a) Engenharia se mostra indispensável para a ampliação da infraestrutura, para a melhoria na qualidade de serviços prestados à sociedade e para a resolução de problemas de caráter econômico e social.
b) Por meio de conhecimentos diversos como administração, economia e engenharia, esses profissionais aperfeiçoam técnicas e táticas de produção assim como organizam atividades financeiras, logísticas e comerciais de uma empresa
c) Numa tentativa de contornar a crise de software e dar um tratamento de engenharia, mais sistemático e controlado, ao desenvolvimento de sistemas de softwares complexos

d) Por estar intimamente ligada a tecnologia, a Engenharia é de suma importância para o mundo moderno, onde é responsável desde o mais simples circuito eletrônico encontrado nas mais variadas tecnologias até os campos de geração, transmissão e distribuição de energia
e) Nenhuma das alternativas

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 2: Tema - 1. FUNDAMENTOS DE SOFTWARE E GERENCIAMENTO DE PROJETOS

3 Objetivos

Compreender as atividades de Gerenciamento de projetos que devem ser incorporadas ao processo de desenvolvimento de software.

4 Tópicos

1.3 O DESENVOLVIMENTO DE SOFTWARE COMO UM PROJETO E AS ATIVIDADES DE GERENCIAMENTO
1.4 PLANEJAMENTO E GERENCIAMENTO DE RISCOS

5 Procedimentos de ensino-aprendizagem

Nesta aula, será exposto o conceito de projeto e será realizada a diferenciação de operações rotineiras. Na sequência o aluno será apresentado às técnicas, habilidades e ferramentas necessárias para o gerenciamento de projetos, tendo maior ênfase no gerenciamento de riscos.

- Situação-problema:
Se você deseja aprender um novo idioma e de vez em quando assiste algumas aulas na internet, ou faz algumas leituras esporádicas, isso pode ser considerado um projeto pessoal? Suponha agora que você tenha planejado estudar duas horas por dia, três vezes na semana durante 18 meses, e após esse período irá fazer uma viagem de 30 dias para exercitar a conversação. Isso pode ser considerado um projeto pessoal?

- Metodologia:
O professor deve iniciar a aula definindo o que é projeto, enfatizando, ainda relacionado ao problema anterior, que para algo ser considerado um projeto, deve haver planejamento de diversos aspectos, como escopo, cronograma e custo, para que ele tenha uma maior chance de sucesso. Na sequência pode-se explicar as definições de gerenciamento de projetos e a aplicação do PMBOK 6. Ainda sobre este tema, o docente pode apresentar o conceito da tripla restrição e na sequência apresentar as outras 7 áreas de conhecimento do PMBOK, juntamente com os seus 5 grupos de processo. Deve-se enfatizar para os alunos, que eles terão uma outra disciplina que aprofundará este tema. O objetivo desta aula é aplicar os conceitos de gerenciamento de projetos na construção de um software. O professor deve adiantar para turma, que o gerenciamento de projetos pode ser combinado com metodologias ágeis, que veremos ainda nesta disciplina, e na sequência apresentar o vídeo [1]. Para

não tomar muito tempo da aula, o docente poderá reproduzi-lo na velocidade 1,5x. Após o vídeo, o professor deve introduzir os conceitos de gerenciamento de riscos e detalhar as etapas de Identificação; Análise; Planejamento e Monitoração dos riscos, segundo Sommerville [2].

- Atividade verificadora de aprendizagem:
Em grupos, os alunos deverão identificar uma lista com 5 riscos para o desenvolvimento de um aplicativo iOS e Android, realizar a análise de cada um deles e planejar as respostas aos riscos. Os trabalhos deverão ser entregues ao professor em aula.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "PMBOK x ÁGIL qual a diferença" disponível em: https://www.youtube.com/watch?v=zFH4PNDZV0M

[2] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011.Páginas 607 até 615.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

8 Aprenda +

- Vídeo "PMBOK® Guide 6a Edição Explicado com Ricardo Vargas" disponível em: https://www.youtube.com/watch?v=rvDnS_wWwJs

- LIMA, Fabiana. Gerenciamento do tempo em projetos de software usando PMBOK. Disponível em: https://www.devmedia.com.br/gerenciamento-do-tempo-em-projetos-de-software-usando-pmbok/32920

- MONTES, Eduardo. Gerenciamento dos riscos: O que é, objetivo e processos. Disponível em: https://escritoriodeprojetos.com.br/gerenciamento-dos-riscos-do-projeto

Atividade Autônoma Aura:

1) O Guia do Conhecimento em Gerenciamento de Projetos (PMBOK) é uma das principais referências na área de gerenciamento de projetos. De acordo com este guia, os processos desta área são organizados nos seguintes grupos:
a) iniciação, criação, execução, monitoramento e controle, e encerramento.
b) iniciação, planejamento, execução, monitoramento e controle, e encerramento.
c) iniciação, planejamento, execução, testes e encerramento.
d) iniciação, criação, execução, testes e encerramento.
e) iniciação, criação, planejamento, execução, testes e encerramento.

2) Na análise qualitativa de riscos, cada risco é analisado individualmente. Para que se realize a priorização são estimados dois fatores. São eles:
a) Probabilidade e Impacto
b) Probabilidade e Exposição
c) Impacto e Exposição
d) Ameaça e Impacto
e) Ameaça e Exposição

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 3: Tema - 2. FASES DO DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Compreender a importância de o requisito para o desenvolvimento de software e um processo formal para identificar, analisar, validar e gerenciar os requisitos do sistema.

4 Tópicos

2.1 ENGENHARIA DE REQUISITOS E ANÁLISE DE SISTEMA

5 Procedimentos de ensino-aprendizagem

Nesta aula, será discutida a Engenharia de Requisitos e suas principais classificações.

- Situação-problema:
Quando um cliente deseja um software, como nós sabemos quanto tempo demorará para ser desenvolvido? Como nós descrevemos para os programadores todas as características que devem ser implementadas? De que forma são listadas todas as funcionalidades que o software deve possuir? Resposta: Através dos requisitos!

- Metodologia:
O professor deve iniciar a aula apresentando o conceito geral de requisitos, e os conceitos de requisitos funcionais e não funcionais, seguido pelo vídeo [1]. O docente pode escolher um aplicativo de celular que seja de uso comum dos alunos e pedir que eles listem alguns requisitos, primeiro os funcionais e depois os não funcionais, gerando uma discussão sobre cada um deles. A seguir, deve-se detalhar outras classificações de requisitos, tais como: inversos, voláteis, estáveis, do cliente, do sistema, etc. e as subdivisões dos requisitos não funcionais apresentadas por Sommerville [2]. O professor deve então abordar as formas de elicitação, análise e especificação dos requisitos, assim como, seu gerenciamento e validação, apresentando então o vídeo [3]. Antes de encerrar a parte teórica da aula, o professor deve apresentar à turma o Modelo Cascata, o qual estudaremos ao longo das próximas três aulas.

- Atividade verificadora de aprendizagem:
O professor deve utilizar a ferramenta Kahoot, para elaborar um quiz com perguntas sobre as principais classificações de requisitos e realizar durante o último tempo de aula.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Requisito Funcional e Não Funcional de Software: entenda a diferença."
https://www.youtube.com/watch?v=YLd6AWKVyas

[2] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011. Páginas 85 até 110.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

[3] Vídeo "Esqueça isso e seu projeto estará condenado. (Análise de Requisitos)". Disponível em:
https://www.youtube.com/watch?v=rVbJ7ykuLig

8 Aprenda +

- Trabalhando com Engenharia de Requisitos disponível em:
https://www.devmedia.com.br/trabalhando-com-engenharia-de-requisitos/30207

- O que são Requisitos Funcionais e Requisitos Não Funcionais?. Disponível em:
https://codificar.com.br/requisitos-funcionais-nao-funcionais/

Atividade Autônoma Aura:

1) São requisitos que tratam das restrições do software, visando sempre a qualidade:
a) Requisitos Inversos
b) Requisitos Voláteis
c) Requisitos Não Funcionais

d) Requisitos Funcionais
e) Requisitos Estáveis

2) São requisitos que definem o que nunca deve ocorrer durante a execução do software:
a) Requisitos de Cliente
b) Requisitos Inversos

c) Requisitos Não Funcionais
d) Requisitos Negativos
e) Requisitos de Domínio

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 4: Tema - 2. FASES DO DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Conhecer as conceitos, elementos e procedimentos centrais da atividade de análise e projeto de sistemas

4 Tópicos

2.2 PROJETO DO SISTEMA

5 Procedimentos de ensino-aprendizagem

Nesta aula, serão diferenciados os conceitos de análise e projeto de sistemas.

- Situação-problema:
Quando um cliente explica o problema que o software deve solucionar, o que o analista de TI deve se preocupar em anotar: o que sistema irá fazer? Ou como o sistema irá fazer? Resposta: o que o sistema irá fazer.

- Metodologia:
O professor deve iniciar o conteúdo dessa aula comparando as definições de análise e projeto de software. Durante a fase de análise o engenheiro de software deve compreender o que o software irá fazer, sem preocupar-se com as tecnologias necessárias para isso. Já durante o projeto, deve-se pensar em uma arquitetura que envolva tecnologias que consigam entregar o resultado esperado. Nesse momento o professor deve retornar ao modelo Cascata apresentado na aula anterior, e explicar que durante a primeira etapa deve-se compreender as funcionalidades do sistema e escrever os requisitos (entendendo o problema) e na segunda etapa, criar diagramas, definir tecnologias, linguagens de programação, etc. (planejando a solução). Como provavelmente, a maior parte dos alunos ainda não teve contato com a UML, o professor pode apresentar rapidamente os diagramas de Classes (nesse momento o professor deve apresentar o vídeo [1]) e Casos de Uso (Vídeo [2]), juntamente com uma breve apresentação de orientação a objetos.

- Atividade verificadora de aprendizagem:
Em grupos, criem cinco requisitos funcionais para um aplicativo de delivery de comida e na sequência desenhem um diagrama de casos de uso. O trabalho deverá ser feito em sala e ao final, entregue ao professor.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Tutorial de Diagramas de Classes UML". Disponível em: https://www.youtube.com/watch?v=rDidOn6KN9k

[2] Vídeo "Tutorial de Caso de Uso UML". Disponível em: https://www.youtube.com/watch?v=ab6eDdwS3rA

[2] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011. Páginas 147 até 154.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

8 Aprenda +

- PINTO, Hudson. Atividades básicas ao processo de desenvolvimento de Software. Disponível em: https://www.devmedia.com.br/atividades-basicas-ao-processo-de-desenvolvimento-de-software/5413

- UML: Diagrama de Casos de Uso. Disponível em: https://medium.com/operacionalti/uml-diagrama-de-casos-de-uso-29f4358ce4d5

Atividade Autônoma Aura:

1) O modelo cascata possui diversos estágios, dentre eles o Projeto de sistema e software, que:
a) É a fase mais longa do ciclo de vida do sistema. O sistema é instalado e colocado em uso e é efetuada a correção de erros que não foram descobertos em estágios iniciais.
b) estabelece os serviços, restrições e metas do sistema por meio de consulta aos usuários, e em seguida define em detalhes estes itens para serem utilizados como uma especificação do sistema.
c) integra as unidades do programa ou programas como um sistema completo e efetuam testes para garantir que os requisitos do software tenham sido atendidos.
d) aloca os requisitos tanto para sistemas de hardware como para sistemas de software, por meio de uma arquitetura geral do sistema.
e) desenvolve o software como um conjunto de programas ou unidades de programas para posteriormente serem aplicados os testes unitários.

2) No modelo de ciclo de vida em cascata:
a) a ideia principal é criar um protótipo executável e por meio de transformações sucessivas chegamos a um sistema implementado totalmente.
b) a ênfase se dá em uma comunicação estreita com o cliente durante o desenvolvimento do produto de software.
c) envolve a análise dos riscos envolvidos no desenvolvimento dos requisitos identificados para produto de software.

d) é recomendável a geração de versões incompletas do sistema as quais podem ser passadas para o usuário final, o qual permite o feedback do processo de desenvolvimento.
e) enfatiza a realização sequencial das atividades do desenvolvimento de um produto de software.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 5: Tema - 2. FASES DO DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Compreender a importância de testes de software, como forma de aferir a qualidade do produto gerado.

4 Tópicos

2.3 IMPLEMENTAÇÃO E TESTES

5 Procedimentos de ensino-aprendizagem

Nesta aula, serão expostos os conceitos de implementação e iniciaremos os estudos dos testes.

- Situação-problema:
Imagine que estamos desenvolvendo um software responsável por realizar cirurgias com robôs e iremos colocá-lo em produção em breve. Como podemos garantir que esse sistema é seguro e não irá apresentar falhas?

- Metodologia:
O professor deve iniciar a aula apresentando novamente o modelo cascata de Sommerville e informar que dará continuidade nas etapas de implementação e testes. Começando pela implementação, o docente deve apresentar o seu conceito e as principais linguagens de programação que estão sendo utilizadas na atualidade, juntamente com os principais SGBDs. Na sequência deverá apresentar o conceito de Casos de Teste e Test Driven Development, onde os testes são desenvolvidos antes das funcionalidades, para direcionar o correto desenvolvimento. Neste momento, o professor deve apresentar o vídeo [1] que irá contextualizar o tema para os alunos. O docente pode apresentar as quatro estratégias de teste Pressman e Sommerville: teste unitário, teste de integração, teste de validação e teste de sistema. Ele não deve esquecer de detalhar o teste de validação, explicando os testes alpha e beta. Para finalizar, os conceitos de verificação e validação devem ser diferenciados e o funcionamento da revisão técnica formal deve ser explicado. Na próxima aula continuaremos abordando testes de software.

- Atividade verificadora de aprendizagem:
Em grupos, os alunos deverão escolher um sistema que seja do conhecimento de todos (rede social, aplicativo de mensagens, delivery, internet banking, etc.) e listar 10 casos de teste, que podem abordar

diversos cenários. Após o trabalho, os grupos deverão apresentar os seus casos de teste, para os demais alunos. Este trabalho vale 1,0 ponto, para compor a nota da AV1.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Estratégias de Teste de Software: O Case WhatsApp". Disponível em: https://www.youtube.com/watch?v=id8Yf3iXroY

[2] PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Porto Alegre: AGMH, 2016. Páginas 466 a 480. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788580555349/

8 Aprenda +

- SANTOS, Glyciane. Melhores práticas na elaboração de casos de teste. Disponível em: https://blog.cedrotech.com/melhores-praticas-na-elaboracao-de-casos-de-teste/

- FERNANDES, Mateus. Teste de Unidade e Teste de Integração: O que são? Disponível em: https://medium.com/@mateus1198/teste-de-unidade-e-teste-de-integração-o-que-são-de58d7a3d3d2

Atividade Autônoma Aura:

1) Dentre os SGBDs abaixo, selecione a alternativa que possui apenas sistemas de código aberto.
a) MongoDB, SQLite, MariaDB, SQL Server
b) SQLite, MariaDB, DB2 e MySQL
c) PostgreSQL, SQLite, MySQL e Access
d) Oracle, SQLite, MariaDB e MongoDB
e) MySQL, PostgreSQL, MongoDB e SQLite

2) João é um Engenheiro de Software que foi incumbido de realizar um conjunto de testes dentro de uma estratégia que foca o esforço de verificação no menor elemento de projeto de um software. Assinale a alternativa que indica qual o tipo de estratégia utilizada por João:
a) Teste de integração.
b) Teste de unidade.
c) Teste de validação.
d) Teste de sistema.
e) Teste de depuração.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 6: Tema - 2. FASES DO DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Distinguir os processos de desenvolvimento de software, com base nos seus métodos e procedimentos, para aprender a modelar e estruturar o desenvolvimento de sistemas.

4 Tópicos

2.3 IMPLEMENTAÇÃO E TESTES
2.4 MANUTENÇÃO

5 Procedimentos de ensino-aprendizagem

Nesta aula, daremos continuidade ao tema testes de software a veremos também conceitos de manutenção.

- Situação-problema:
Exceto pelo teste unitário, que é criado pelo próprio desenvolvedor durante a codificação, é comum que as empresas tenham outra área responsável por desenvolver os testes. Qual é a importância desses testes? Todos os testes têm o mesmo objetivo? Na aula de hoje, veremos que não.

- Metodologia:
O professor deve iniciar a aula relembrando conceitos discutidos na semana anterior e na sequência apresentar os principais tipos de teste, entre elas testes de carga, estresse, usabilidade, segurança, recuperação e regressão. Logo depois deve explicar que existem duas principais técnicas, os testes de caixa branca, também conhecidos como estruturais e os testes de caixa preta, também chamados de testes funcionais. O primeiro grupo, constróis seus casos de teste conhecendo o código fonte do software, no segundo, tudo que se sabe são as regras de negócio e os requisitos. Para concluir os estudos dos testes, o professor deve explicar o funcionamento da automatização de testes e citar as principais ferramentas, como o Cucumber e o Selenium. Neste momento o professor deve apresentar o vídeo [1], para fixar os conhecimentos aprendidos. Por fim o docente deve retornar ao modelo cascata e apresentar a sua última etapa, a operação e manutenção. Nela teremos quatro tipos de manutenções, corretiva, adaptativa, perfectiva e preventiva, cada uma com suas características e especificidades.

- Atividade verificadora de aprendizagem:
Em grupos, os alunos deverão criar um mapa mental incluindo todos os conceitos aprendidos sobre

testes de software. Para isso, poderão utilizar a ferramenta Miro, ou outra da preferência do grupo. Este trabalho vale 1,0 ponto, para compor a nota da AV1.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Você ainda faz testes de software manuais?". Disponível em: https://www.youtube.com/watch?v=fxL04i00jJw

[2] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011.Páginas 203 até 230.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

8 Aprenda +

- VIEGAS, Júlio. Teste de Software: Introdução, Conceitos Básicos e Tipos de Testes. Disponível em: https://blog.onedaytesting.com.br/teste-de-software/

- Teste de Software: O Que é, Porque é Importante e Principais Tipos. Disponível em: https://blog.tecnospeed.com.br/teste-de-software/

Atividade Autônoma Aura:

1) Complete as lacunas com Testes de Validação ou Testes de Verificação.
_____ são considerados Testes Caixa preta. _____ são considerados Testes Caixa Branca. Para realizar_____ é necessário ter acesso ao código fonte. Os _____ testam as funcionalidades com a finalidade de garantir que os requisitos do software construído atendem a necessidade do cliente.
a) Testes de Validação, Testes de Verificação, Testes de Validação, Testes de Validação
b) Testes de Validação, Testes de Validação, Testes de Verificação, Testes de Validação
c) Testes de Validação, Testes de Verificação, Testes de Verificação, Testes de Validação
d) Testes de Verificação, Testes de Validação, Testes de Verificação, Testes de Validação
e) Testes de Verificação, Testes de Validação, Testes de Validação, Testes de Verificação

2) A adaptação no software para acomodar as constantes mudanças ocorridas em seu ambiente externo, segundo Pressman é considerada uma mudança
a) Corretiva
b) Adaptativa
c) Evolutiva
d) Preventiva
e) Perfectiva

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 7: Tema - 3. MODELOS DE PROCESSOS DE DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Compreender as características dos processos prescritivos, ressaltando os modelos Cascata, iterativos, incrementais e evolucionários.

4 Tópicos

3.1 MODELOS DE PROCESSOS PRESCRITIVOS

5 Procedimentos de ensino-aprendizagem

Nesta aula, serão apresentados outros processos para o desenvolvimento de software

- Situação-problema:
Nas aulas anteriores, conhecemos todas as etapas do modelo cascata, também chamado de modelo clássico. Você acredita que um dos primeiros modelos de processo para o desenvolvimento de software, ainda seja amplamente utilizado pelas empresas? Por quê? Quais os seus principais problemas?

- Metodologia:
O professor deve iniciar a aula descrevendo o contexto histórico do modelo cascata e apresentando seus principais problemas. Na sequência deve apresentar os modelos que foram usados para tentar otimizá-lo. Iniciando pelo modelo em cascata com retroalimentação, que tentou minimizar o impacto das mudanças de requisitos e o modelo V, que desenhou uma estratégia de testes durante todo desenvolvimento, depois o Modelo incremental, adequado quando os requisitos são conhecidos no início do desenvolvimento e existe necessidade de entrega de
um produto funcional em pouco tempo. Na sequência os modelos evolucionários (Iterativos), em que os requisitos são parcialmente definidos e depois são refinados. Como é o exemplo da prototipação, que constrói protótipos descartáveis e o modelo espiral, o qual evolui de maneira cíclica, utilizando a prototipagem e a análise de riscos. Para melhor fixar estes conceitos o professor deve apresentar o vídeo [1] para a turma.

- Atividade verificadora de aprendizagem:
O professor deverá dividir a turma em grupos, e sortear um Modelo de Ciclo de Vida para cada um deles. Após 15 minutos de pesquisa e organização, o docente deverá promover uma discussão e cada

grupo ficará responsável em defender as características do seu modelo.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Ciclo de vida dos Software". Disponível em: https://www.youtube.com/watch?v=6_fVcpC0cxE

[2] PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Porto Alegre: AGMH, 2016. Páginas 41 até 51. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788580555349/

8 Aprenda +

- DIAS, Ricardo. O Modelo em Espiral de Boehm. Disponível em: https://medium.com/contexto-delimitado/o-modelo-em-espiral-de-boehm-ed1d85b7df

- MEDEIROS, Igor. Introdução aos Processos de Software e o Modelo Incremental e Evolucionário. Disponível em: https://www.devmedia.com.br/introducao-aos-processos-de-software-e-o-modelo-incremental-e-evolucionario/29839

Atividade Autônoma Aura:

1) Tratando-se de desenvolvimento de software, existem diversos modelo de ciclo de vida, tais como cascata, espiral e prototipagem; sendo assim, é correto afirmar que

a) o modelo em espiral normalmente possui um maior potencial de risco, uma vez que ele não lida explicitamente com isso.
b) o modelo de prototipagem deve, necessariamente, descartar os protótipos construídos; dessa forma, ele costuma ser mais custoso.
c) o modelo em cascata possui fases bem definidas, que podem ser desenvolvidas incrementalmente, em diferentes ciclos de desenvolvimento, isto é, a fase seguinte pode ser executada, ainda que a fase anterior não tenha sido finalizada completamente.
d) no modelo em espiral, o software é desenvolvido em apenas uma iteração.
e) o modelo em cascata possuem fases bem definidas e executadas sequencialmente. Além disso, não há sobreposição entre as fases, isto é, a fase seguinte somente pode ser executada após a finalização da fase anterior.

2) Assinale a alternativa que apresenta o modelo de desenvolvimento de software aplicado em situações onde o cliente apenas define objetivos gerais do sistema sem identificar detalhadamente os requisitos de entrada e saída.
a) RAD.
b) Incremental.
c) Prototipagem.
d) Espiral.

e) Cascata

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 8: Tema - 3. MODELOS DE PROCESSOS DE DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Apresentar o Processo Unificado, idealizado pela Rational, baseado na orientação a objetos e a UML (Linguagem de modelagem unificada).

4 Tópicos

3.2 O PROCESSO UNIFICADO

5 Procedimentos de ensino-aprendizagem

Nesta aula, será apresentado o Modelo Unificado, também conhecido com RUP.

- Situação-problema:
Nos modelos apresentados anteriormente, tínhamos uma fase bem definida para fazer cada coisa. Mas e se existisse um modelo que se preocupasse com modelagem de negócios, análise de requisitos, implementação e outras disciplinas durante todo o processo de desenvolvimento? Esse modelo existe, se chama RUP e aprenderemos mais sobre ele nesta aula.

- Metodologia:
O professor deve iniciar a aula contextualizando o momento histórico que o RUP foi criado e a sua posterior aquisição pela IBM. Durante a apresentação do modelo o professor deve utilizar o gráfico que apresenta as dez disciplinas no eixo vertical e as fases no eixo horizontal. O tamanho da área preenchida em cada disciplina representa o volume de trabalho que será aplicado em determinada disciplina durante uma etapa. O docente pode destacar que no início do projeto o esforço para gerenciamento de requisitos é muito grande e vai diminuindo ao longo do projeto, mas só acaba, quando o sistema está quase pronto. Outro ponto de destaque, são os testes, que são iniciados antes mesmo da implementação das funcionalidades, o que sugere a aplicação da técnica de TDD. O vídeo [1] deve ser apresentado para facilitar o entendimento dos alunos. Na sequência cada uma das fases e disciplinas deve ser explicada e detalhada.

- Atividade verificadora de aprendizagem:
Utilizando a plataforma MS Forms, o professor deverá criar um quiz, abordando perguntas sobre os principais processos de desenvolvimento de software. Esta atividade vale 1,0 ponto, para compor a nota da AV1.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Aula sobre Modelos de Processos da Engenharia de Software". Disponível em: https://www.youtube.com/watch?v=qX00H-COqmw

[2] PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Porto Alegre: AGMH, 2016. Páginas 55 até 57. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788580555349/

8 Aprenda +

- RUP - Rational Unified Process. Disponível em: https://www.devmedia.com.br/rup-rational-unified-process/4574

- RUP - Primeiros Passos. Disponível em: https://www.tiespecialistas.com.br/rup-primeiros-passos/

Atividade Autônoma Aura:

1) O Processo Unificado de software é uma tentativa de aproveitar os melhores recursos e características dos modelos tradicionais de processo de software. Sobre o Processo Unificado de software, assinale a afirmativa correta.
a) O software é dirigido a casos de uso, centrado na arquitetura, sequencial e incremental.
b) O software é entregue aos usuários finais na fase de transição.
c) Os modelos de caso de uso, análise, projeto e implementação são desenvolvidos na fase de concepção.
d) O planejamento é realizado na fase de elaboração.
e) Os requisitos não funcionais são descritos em um conjunto de casos de uso preliminares.

2) Quais são as quatro fases do RUP (Rational Unified Process)?
a) Analise de requisitos, Testes, Transição e Documentação
b) Iniciação, Elaboração, Construção e Transição
c) Iniciação, desenvolvimento, adaptação e testes
d) Implementação, desenvolvimento, atualização, testes
e) Projeto, Arquitetura, Testes e Ambiente

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 9: Tema - 3. MODELOS DE PROCESSOS DE DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Reconhecer a existência de procedimentos e processos que visam tornar o processo de desenvolvimento mais enxuto, menos burocrático e portanto mais ágil, com base em princípios do Manifesto Ágil.

4 Tópicos

3.3 DESENVOLVIMENTO ÁGIL - EXTREME PROGRAMMING (XP)

5 Procedimentos de ensino-aprendizagem

Nesta aula, serão apresentados os conceito de metodologias ágeis.

- Situação-problema:
Até agora estudamos diversos modelos de processo para desenvolvimento de software. Todos eles com etapas e procedimentos bem rígidos. Mas será que na prática, as organizações buscam por um método tradicional ou um mais flexível? Imagine se pudéssemos aplicar todas as boas práticas aprendidas, gerando o mínimo de burocracia! Hoje conheceremos as metodologias ágeis.

- Metodologia:
O professor deve iniciar a aula apresentando a história do manifesto ágil, contando que no final dos anos 90 todos estavam em busca de uma forma mais leve de desenvolver software e diversas iniciativas começaram a surgir ao redor do mundo. Em fevereiro de 2001, dezessete representantes desses métodos se reuniram em um resort de inverno, buscando em um local descontraído, encontrar um padrão para o desenvolvimento de software com maior leveza e agilidade. Juntos criaram um documento e deram o nome de Manifesto Ágil, contendo quatro valores e doze princípios que orientam as equipes de como desenvolver software de maneira leve. Para fixar os conhecimentos o professor deve apresentar o vídeo [1], contendo mais detalhes sobre este assunto. Após o vídeo, deve-se informar aos alunos, que nesta aula aprenderemos mais sobre a XP e na semana que vem trataremos de outras metodologias ágeis. Na XP, eXtreme Programming, o objetivo é levar todas as boas práticas ao extremo. O docente deve explicar os valores, princípios e práticas listados pela metodologia. Por fim exibir o vídeo [2] para enfatizar a aplicação dos métodos ágeis .

- Atividade verificadora de aprendizagem:

Em grupos, os alunos deverão ler o manifesto ágil, disponível em https://agilemanifesto.org/iso/ptbr/manifesto.html , preparar um resumo do que entenderam e entregar ao professor.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Metodologia Ágil 2020: O que é Metodologia Ágil? Entenda o que são MÉTODOS ÁGEIS em 8 MINUTOS ". Disponível em: https://www.youtube.com/watch?v=cT_X4_n0NJ4

[2] Vídeo "Metodologia Agile #1AG - O manifesto que revolucionou o relacionamento entre times e clientes". Disponível em: https://www.youtube.com/watch?v=aNDritqmhps

[3] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011.Páginas 57 até 72.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

8 Aprenda +

- BECK Kent ; et al. Manifesto para Desenvolvimento Ágil de Software. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html

- Um pouco sobre o Método XP (Extreme Programming). Disponível em: https://medium.com/codengage/um-pouco-sobre-o-método-xp-ea2e6baae561

Atividade Autônoma Aura:

1) Sobre o Manifesto Ágil, avalie as afirmativas a seguir:
I. Foi um encontro realizado para unificar diversos métodos em uma única metodologia ágil.
II. Foi muito utilizado por anos, mas teve o seu fim com a chegada do Scrum.
III. Foi elaborado por profissionais respeitados em todo o mundo incluindo criadores e representantes de diversos "métodos leves".
Está correto o que se afirma em:
a) I, apenas.
b) II, apenas.
c) III, apenas.
d) I e II.
e) II e III.

2) Em projetos de desenvolvimento de software, a extreme programming (XP) é um método ágil que usa a prática de:
a) projetos com planejamento completo sem incrementos.
b) grandes releases.
c) grande quantidade de horas extras.
d) trabalho em pares de desenvolvedores.

e) integrações após a entrega do software completo

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 10: Tema - 3. MODELOS DE PROCESSOS DE DESENVOLVIMENTO DE SOFTWARE

3 Objetivos

Conhecer diversas metodologias ágeis, e saber as principais vantagens e desvantagens de cada uma delas.

4 Tópicos

3.4 ESENVOLVIMENTO ÁGIL - SCRUM E PROCESSO UNIFICADO ÁGIL

5 Procedimentos de ensino-aprendizagem

Nesta aula, daremos continuidade ao conceito de metodologias ágeis, abordando o Scrum e outras metodologias.

- Situação-problema:
Você consegue imaginar uma metodologia ágil, que é tão eficiente para o desenvolvimento de softwares que pode ser usada para gerenciar qualquer projeto?

- Metodologia:
O professor deve iniciar a aula apresentando a história do surgimento do Scrum e a analogia do seu nome com uma jogada de rugby, onde o time que for mais unido vence. Então deve explicar o funcionamento do time Scrum e os seus papeis, o time-box denominado Sprint, os seus eventos e artefatos. Para melhor compreensão da turma, o vídeo [1] deverá ser exibido antes de prosseguir com o conteúdo. Para demonstrar a importância deste assunto, o professor pode escolher algumas questões de concursos recentes e solucionar em conjunto com a turma antes de seguir para o próximo assunto. Na sequência, o docente deve apresentar rapidamente outras metodologias como o processo unificado ágil, DSDM (Modelo de desenvolvimento de sistemas dinâmicos), ASD (Crystal Desenvolvimento de software Adaptativos), FDD (Desenvolvimento dirigido a funcionalidade) e LSD (Desenvolvimento de software enxuto). Sempre apresentando as principais características, vantagens e desvantagens.

- Atividade verificadora de aprendizagem:
Em grupos, os alunos deverão criar um mapa mental incluindo todos as metodologias ágeis aprendidas. Para isso, poderão utilizar a ferramenta Miro, ou outra da preferência do grupo.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Scrum - Aprenda Scrum em 9 minutos". Disponível em: https://www.youtube.com/watch?v=XfvQWnRgxG0

[2] PRESSMAN, Roger; MAXIM, Bruce. Engenharia de Software. Porto Alegre: AGMH, 2016. Páginas 66 até 84. Disponível em: https://integrada.minhabiblioteca.com.br/#/books/9788580555349/

8 Aprenda +

- Guia do Scrum. Disponível em: https://www.scrumguides.org/docs/scrumguide/v1/Scrum-Guide-Portuguese-BR.pdf

- Modelos de Processos Ágeis: conceitos e princípios. Disponível em: https://www.devmedia.com.br/modelos-de-processos-ageis-conceitos-e-principios/30059

Atividade Autônoma Aura:

1) Sobre metodologias ágeis, avalie as afirmativas a seguir:

I. O Scrum é uma metodologia ágil que pode ser utilizada em projetos de diversas áreas além de TI.

II. A XP é uma metodologia ágil mundialmente conhecida.

III. O PMBOK é a metodologia ágil mais usada para gerenciamento de projetos no mundo.

Está correto o que se afirma em:

a) I, apenas.

b) II, apenas.

c) III, apenas.

d) I e II.

e) II e III.

2) A Scrum é um framework muito utilizado nas organizações, PORQUE funciona bem com equipes de todos os tamanhos.

a) as duas afirmações são verdadeiras e a segunda justifica a primeira.

b) as duas afirmações são verdadeiras e a segunda não justifica a primeira.

c) a primeira afirmação é verdadeira e a segunda é falsa.

d) a primeira afirmação é falsa e a segunda é verdadeira.

e) as duas afirmações são falsas.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 11: Tema - 4. QUALIDADE DE SOFTWARE (ATIVIDADE PRÁTICA SUPERVISIONADA)

3 Objetivos

Conhecer os conceitos gerais das normas e padrões de qualidade de software.

4 Tópicos

4.1 QUALIDADE DE PROCESSO E DE PRODUTO DE SOFTWARE

5 Procedimentos de ensino-aprendizagem

Nesta aula, estaremos conectados com o conteúdo digital. O aluno explora e estuda, previamente, o conteúdo digital disponível em seu ambiente virtual.

6 Recursos didáticos

A aula será realizada no ambiente virtual de aprendizagem.

7 Leitura específica

O aluno deverá consultar a bibliografia proposta no tema.

8 Aprenda +

O aluno deverá aprofundar os seus estudos navegando no explore + disponível no tema digital.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 12: Tema - 4. QUALIDADE DE SOFTWARE (ATIVIDADE PRÁTICA SUPERVISIONADA)

3 Objetivos

Compreender o processo de gerenciamento da qualidade e as atividades centrais do processo de garantia da qualidade de software.

4 Tópicos

4.2 PROCESSO DA GARANTIA DE QUALIDADE DE SOFTWARE

5 Procedimentos de ensino-aprendizagem

Nesta aula, estaremos conectados com o conteúdo digital. O aluno explora e estuda, previamente, o conteúdo digital disponível em seu ambiente virtual.

6 Recursos didáticos

A aula será realizada no ambiente virtual de aprendizagem.

7 Leitura específica

O aluno deverá consultar a bibliografia proposta no tema.

8 Aprenda +

O aluno deverá aprofundar os seus estudos navegando no explore + disponível no tema digital.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 13: Tema - 4. QUALIDADE DE SOFTWARE (ATIVIDADE PRÁTICA SUPERVISIONADA)

3 Objetivos

Conhecer os principais conceitos de planejamento e controle da qualidade.

4 Tópicos

4.3 PLANEJAMENTO DA QUALIDADE E O CONTROLE DA QUALIDADE DE SOFTWARE

5 Procedimentos de ensino-aprendizagem

Nesta aula, estaremos conectados com o conteúdo digital. O aluno explora e estuda, previamente, o conteúdo digital disponível em seu ambiente virtual.

6 Recursos didáticos

A aula será realizada no ambiente virtual de aprendizagem.

7 Leitura específica

O aluno deverá consultar a bibliografia proposta no tema.

8 Aprenda +

O aluno deverá aprofundar os seus estudos navegando no explore + disponível no tema digital.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 14: Tema - 4. QUALIDADE DE SOFTWARE (ATIVIDADE PRÁTICA SUPERVISIONADA)

3 Objetivos

Compreender o que são métricas de software e sua relevância na avaliação de determinados atributos da qualidade.

4 Tópicos

4.4 MEDIÇÕES E MÉTRICAS DO SOFTWARE

5 Procedimentos de ensino-aprendizagem

Nesta aula, estaremos conectados com o conteúdo digital. O aluno explora e estuda, previamente, o conteúdo digital disponível em seu ambiente virtual.

6 Recursos didáticos

A aula será realizada no ambiente virtual de aprendizagem.

7 Leitura específica

O aluno deverá consultar a bibliografia proposta no tema.

8 Aprenda +

O aluno deverá aprofundar os seus estudos navegando no explore + disponível no tema digital.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 15: Tema - 5. GERENCIAMENTO DE CONFIGURAÇÕES

3 Objetivos

Compreender a importância de gerenciamento de configurações para sistemas grandes e/ou complexos.

4 Tópicos

5.1 FUNDAMENTOS DE GERENCIAMENTO DE CONFIGURAÇÕES
5.2 GERENCIAMENTO DE MUDANÇAS, VERSÕES E RELEASES

5 Procedimentos de ensino-aprendizagem

Nesta aula, será apresentado o gerenciamento da configuração e seus conceitos.

- Situação-problema:
Sabemos que um software pode utilizar muitos componentes e eles podem ser atualizados de maneira independente. Mas como é possível realizar o controle de todas as versões das diversas partes de um software? Como manter um controle de quais funcionalidades eram entregues em cada uma das versões do sistema?

- Metodologia:
O professor deve iniciar a aula apresentando uma visão geral do gerenciamento da configuração, para proporcionar a todos da turma, principalmente os que nunca atuaram no mercado de trabalho, um entendimento comum sobre o tema. Então deve-se explicar a correlação com o gerenciamento de mudanças, gerenciamento de versões e gerenciamento de releases. Para isso, o docente pode se basear no livro do Sommerville, capítulo 25 [1]. Deve-se ainda explicar o que é item de configuração, baseline e controle de configuração. Por fim o professor deve apresentar o que é, e como funciona, o repositório Git, apresentando o vídeo [2], para melhor ilustrar esta ferramenta.

- Atividade verificadora de aprendizagem:
Em grupos, os alunos deverão pesquisar quais os principais comandos do Git, e entregar um resumo para o professor. Caso um laboratório esteja disponível, os alunos deverão exercitar os comandos aprendidos.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] SOMMERVILLE, Ian. Engenharia de Software. 10ª Ed. São Paulo: Pearson Prentice Hall, 2011.Páginas 693 até 614.
Disponível em: https://plataforma.bvirtual.com.br/Leitor/Loader/168127/pdf

[2] Vídeo "Entenda o GIT em 10 MINUTOS". Disponível em: https://www.youtube.com/watch?v=FV-hMoqHtcU

8 Aprenda +

- Gerência de Configuração de Software. Disponível em: https://www.devmedia.com.br/gerencia-de-configuracao-de-software/9145

- O que é Gerência de Configuração de Software? Disponível em: https://blog.pronus.io/posts/o-que-eh-gerencia-de-configuracao-de-software/

Atividade Autônoma Aura:

1) Considerando a gerência de configuração de um projeto de software, um item importante a ser considerado são seus baselines, que visam especificar

a) a equipe responsável pelo desenvolvimento do projeto.

b) a forma como o software deverá ser testado.

c) o ciclo de vida que vai ser seguido no projeto.

d) o cronograma de execução de cada versão do sistema.

e) os componentes (incluindo bibliotecas) e suas versões para a definição de um sistema.

2) O git é um sistema de controle de versão muito utilizado em desenvolvimento de sistemas de software. Sobre o git, é correto afirmar:

a) O comando git push é utilizado para envio das alterações confirmadas no diretório local para o repositório remoto.

b) O comando git clone faz a cópia apenas dos arquivos, sendo as informações do repositório inicializadas como no comando git init.

c) O comando git add faz a confirmação das alterações de forma definitiva.

d) O comando git checkout faz uma atualização do diretório local com o diretório remoto.

e) O comando git add faz a confirmação das alterações de forma definitiva.

Plano de Aula

1 Código e nome da disciplina

ARA0097 ENGENHARIA DE SOFTWARE

2 Semana/Tema

Semana 16: Tema - 5. GERENCIAMENTO DE CONFIGURAÇÕES

3 Objetivos

Compreender de que forma as ferramentas CASE podem apoiar o gerenciamento de configurações.

4 Tópicos

5.3 CONSTRUÇÃO DE SISTEMAS E FERRAMENTAS CASE

5 Procedimentos de ensino-aprendizagem

Nesta aula, será ensinado o que são ferramentas CASE e como elas podem ser utilizadas para apoiar a construção de sistemas.

- Situação-problema:
Você conhece algum software que seja utilizado para ajudar a desenvolver outros software? Será que existe algum software, que sirva para ajudar a aplicar os conhecimentos aprendidos em engenharia de software na construção de sistemas?

- Metodologia:
O professor deve iniciar a aula apresentando a definição de ferramentas CASE, Computer Aided Software Engineering, ou seja, engenharia de software assistida por computador e explicar a importância de automatizar o gerenciamento das boas práticas de engenharia de software. Elas podem ser divididas em Upper CASE, Lower CASE e Integrated CASE. A primeira apóia as etapas iniciais de criação dos sistemas: as fases de planejamento, análise e projeto do programa ou aplicação. A segunda dá apoio a parte física, isto é, a codificação testes e manutenção da aplicação. E a terceira classifica os produtos que cobrem todo o ciclo de vida do software, desde os requisitos do sistema até o controle final da qualidade. O docente pode utilizar alguns exemplos de IDEs ou software de modelagem de sistemas, mas deve destacar as vantagens de se utilizar tais ferramentas, como por exemplo, aumentar a qualidade no produto final, agilizar o tempo para tomada de decisão, diminuir a quantidade de código de programação, reduzir custos na manutenção e aumentar a facilidade do desenvolvimento. Para fixar o conhecimento, o docente deve apresentar os vídeos [1] e [2].

- Atividade verificadora de aprendizagem:
Em grupos, pesquisem na internet três ferramentas case e apresentem para o restante da turma suas principais vantagens e desvantagens.

6 Recursos didáticos

Sala de aula equipada com quadro branco, projetor multimídia, computador com caixa de som, acesso à internet e ao acervo bibliográfico do ambiente virtual.

7 Leitura específica

[1] Vídeo "Computer Aided Software Engineerings". Disponível em: https://www.youtube.com/watch?v=hHvypTnvF5E

[2] Vídeo "Ferramentas de Modelagem de Processos". Disponível em: https://www.youtube.com/watch?v=4kmok40SDzY

[3] COSTA, Juliana. Ferramentas CASE: Conhecendo algumas boas opções. Disponível em: https://www.devmedia.com.br/ferramentas-case-conhecendo-algumas-boas-opcoes/32034

8 Aprenda +

- Ferramenta Case que auxilia o processo de engenharia de software e-commerce. Disponível em: https://www.teo.com.br/2016/09/11/ferramenta-case-que-auxilia-o-processo-de-engenharia-de-software-e-commerce/

Atividade Autônoma Aura:

1) Com relação a ferramentas CASE (Computer-Aided Software Engineering), a sigla I-CASE refere-se:

a) aos produtos que apoiam as etapas iniciais de criação dos sistemas: as fases de planejamento, análise e projeto do programa ou aplicação.

b) à classificação dos produtos que cobrem todo o ciclo de vida do software, desde os requisitos do sistema até o controle final da qualidade.

c) aos produtos que dão apoio à parte física, isto é, a codificação, testes e manutenção da aplicação.

d) aos produtos voltados para o desenvolvimento de aplicações para a Internet.

e) às ferramentas voltadas à especificação da interação usuário-computador (IHC).

2)A tecnologia CASE (Engenharia de Software Auxiliada por Computador) fornece apoio ao processo de software pela automação de algumas atividades de processo. Sobre a classificação funcional de ferramentas CASE, são tipos válidos:

a) Ferramentas de Prototipação e Ferramentas de Teste.

b) Ferramentas de Esquema e Ferramentas de Ortografia.

c) Ferramentas de Semântica e Ferramentas de Arguição.

d) Ferramentas de Escopo e Ferramentas de Sintaxe.

e) Ferramentas de Aspecto e Ferramentas de Coerência.